A Project Report on

TCP vs UDP Chat Application

(Mini Project)

Of Course

Computer Network

Submitted By

Kavyansh Goenka

Roll No:

N248

SAP ID:

70472400283

Under The Guidance Of

Prof. Sonia Relan

Mukesh Patel School of Technology & Management Engineering

Department of Computer Engineering

Shirpur-425405

Year : 2025-26

Abstract

This mini-project provides a practical and visual comparative analysis of the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP). The objective was to implement both protocols in a functional chat application to highlight their fundamental differences: reliability (TCP) versus speed (UDP). The project includes two main modules: a multi-threaded Python con- sole application for real-world peer-to-peer communication, and an interactive HTML/JavaScript visual simulator. The simulator graphically demonstrates concepts like the TCP three-way hand- shake, sequencing, and acknowledgment (ACK), as well as the inherent unreliability of UDP, showcasing packet loss and reordering. The project successfully visualizes the crucial trade- offs network engineers face when selecting a transport layer protocol, serving as an effective educational tool.

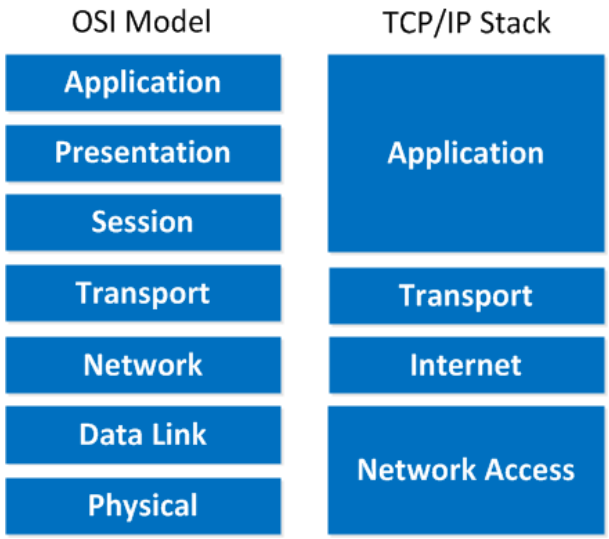
1. Problem Statement

A foundational challenge in computer networking education is the conceptual difference be- tween connection-oriented (TCP) and connectionless (UDP) protocols. Understanding the mech- anisms of reliability (e.g., error checking, flow control, and acknowledgments) versus best-effort delivery is often difficult to grasp purely from theoretical lectures.

The problem addressed by this project is the lack of a simple, side-by-side demonstration that allows users to:

1. Experience the protocols by sending messages through functional chat clients.
2. Visually observe the packet exchange process, including the overhead of TCP (ACKs) and the failure points of UDP (loss/reordering).

The solution is a dual-component application that translates these abstract protocols into observable, interactive events.



2. Project Components, Logic, and Output

2.1 Input

The application accepts user input via two interfaces:

1) Python Terminal (chat_Ai.py):

- User choice of protocol (TCP/UDP).
- Host/Join decision (Server/Client role).
- IP address and port number.
- Text messages for peer-to-peer chat.

2) Web Simulator (tcpudp_chat.html):

a. Protocol selection via a toggle switch (TCP/UDP).

b. Toggle switch to enable/disable simulated packet loss and reordering for UDP.

c. Text message input field.

2.2 Core Logic

The core logic is divided between the two components:

2.2.1 Python Logic (Real-World Chat)

- **Technology:** Python socket module and threading library.
- **Architecture:** Implements concurrent sending and receiving using two separate threads (send_loop and receive_loop) to handle duplex communication.
- **TCP (SOCK_STREAM):** Uses socket.accept() (Host) and socket.connect() (Client) for con- nection establishment. Data is sent using sock.sendall().
- **UDP (SOCK_DGRAM):** Uses socket.bind() (Host) but is connectionless. The Host must first receive a packet to discover the Client's address (client_addr), which is then used for sub- sequent sock.sendto(msg.encode(), peer_addr) calls.

Note on Code Generation: The Python code for the multi-threaded chat application was generated with the assistance of an Artificial Intelligence (AI) model, which helped structure the socket, threading, and host/client logic efficiently.

2.2.2 HTML/JavaScript Logic (Visual Simulation)

- **Technology:** HTML5, Tailwind CSS, and pure JavaScript for animation.
- **Packet Representation:** Messages are split into individual ”packets” (represented as small animated circles with sequence numbers).
- **TCP Simulation:** Simulates sequential packet transmission. After a packet arrives at the server (destination), a corresponding blue ACK (acknowledgment) packet travels back to the client (source) before the next data packet is sent. This visually represents the hand- shake overhead and ordered delivery.
- **UDP Simulation:** Packets are sent concurrently without waiting for any acknowledgment. When packet loss is enabled, JavaScript's Math.random() function randomly drops packets or applies a variable delay (reorderDelay) to simulate reordering upon arrival.

3. Output

1. **Python Output**: Standard console output displaying real-time chat messages prefixed by the sender's role ([HOST] ->, [CLIENT] ->).
2. **HTML/Simulator Output:**
   - **Visual Animation:** Packets move across a simulated network path between Client and Server nodes.
   - **Chat Box:** Displays the assembled messages, clearly indicating if a UDP message was incomplete due to loss.
   - **Network Event Log:** A timestamped log detailing every action, including packets sent, packets received, ACKs sent/received, and instances of packet loss.

4. Execution

4.1 Python Chat Execution

The chat_Ai.py file is executed in a terminal. Two instances must be run concurrently, one as the Host and one as the Join (Client).

- **TCP Mode:** Once connected, communication is stable and reliable.
- **UDP Mode:** Communication starts after the client sends the initial message. Messages are delivered quickly, but unlike TCP, termination is done simply by exiting the loop, as there is no formal connection closure.

4.2 Visual Simulation Execution

The tcpudp_chat.html file is opened in a web browser.

1. **Default (TCP) Test:** The user types a message and observes the slow, sequential process involving data packets and ACK packets, guaranteeing all parts arrive in order.
2. **UDP with Loss Disabled:** The user switches the toggle to UDP. Messages are observed to travel without ACK overhead, resulting in faster visual transfer.
3. **UDP with Loss Enabled**: The user enables the Packet Loss & Reordering switch. When sending a long message, some packets fail to reach the server, resulting in an incomplete message displayed in the chat box, clearly illustrating UDP's unreliability.

5. Outcome and Learnings

5.1 Outcome

The project successfully delivered a robust and educational toolkit.

1. **Protocol Confirmation:** The Python application confirms that both protocols can success- fully facilitate real-time chat, demonstrating the core transport functionality.
2. **Visual Proof of Concepts:** The HTML simulator visually and dramatically proves the theoretical differences:
    - **TCP:** Guaranteed delivery through a sequential, feedback-driven (ACK) mechanism, incurring higher overhead (more steps/slower perceived speed).
    - **UDP:** Faster, "fire-and-forget" approach with minimal overhead, but confirmed to be vulnerable to data loss and message reordering.

5.2 Learnings

The following key concepts were solidified through the development and testing of this project:

- **Trade-off between Reliability and Speed:** The most significant learning is that protocol selection is a trade-off. TCP is ideal for file transfers and chat (where integrity is critical), while UDP is better for streaming video/audio or gaming.
- **Stateless vs. Connection-Oriented: The** implementation highlighted the complexity of managing state in TCP (connection setup, closing, and sequencing) versus the simple, stateless nature of UDP.
- **Asynchronous Programming:** Threading was essential in the Python application to pre- vent the application from blocking while waiting for input or data, a necessary component of any real-time network application.

Input:

chat_Ai.py - C:\Users\Kavyansh Goenka\Downloads\CN Proj\chat_Ai.py (3.13.7)

File Edit Format Run Options Window Help

```
import socket
import threading
from contextlib import suppress
import time

BUFFER = 4096

# -------------------- Chat Functions --------------------
def receive_loop(sock, stop_event, role, peer_role, protocol="TCP", peer_addr=None):
    """Receive loop for TCP or UDP"""
    try:
        while not stop_event.is_set():
            if protocol == "TCP":
                data = sock.recv(BUFFER)
                if not data:
                    stop_event.set()
                    print(f"\n[{peer_role}] disconnected.")
                    break
            else:  # UDP
                data, _ = sock.recvfrom(BUFFER)
                if not data:
                    continue

            print(f"[{peer_role}] -> {data.decode().strip()} \n")

    except Exception:
        stop_event.set()

def send_loop(sock, stop_event, role, peer_role, protocol="TCP", peer_addr=None):
    """Send loop for TCP or UDP using input()."""
    try:
        while not stop_event.is_set():
            msg = input(f"{role} -> {peer_role}: \n")
            if msg.lower() in ("exit", "quit"):
                stop_event.set()
                if protocol == "TCP":
                    with suppress(Exception):
                        sock.shutdown(socket.SHUT_RDWR)
                break

            if protocol == "TCP":
                sock.sendall(msg.encode())
            else:
                sock.sendto(msg.encode(), peer_addr)

    except (KeyboardInterrupt, EOFError):
        print("\nExiting...")
        stop_event.set()
    except Exception:
        stop_event.set()

# -------------------- TCP --------------------
def tcp_host():
```

chat_Ai.py - C:\Users\Kavyansh Goenka\Downloads\CN Proj\chat_Ai.py (3.13.7)

File Edit Format Run Options Window Help

```
    """Starts a TCP server and waits for a client to connect."""
    host = input("Bind host [0.0.0.0]: ") or "0.0.0.0"
    port = int(input("Port [5000]: ") or 5000)

    srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((host, port))
    srv.listen(1)
    print(f"[TCP HOST] Listening on {host}:{port} ...")
    conn, addr = srv.accept()
    print(f"[TCP HOST] Connected by {addr}")

    stop_event = threading.Event()
    threading.Thread(target=receive_loop, args=(conn, stop_event, "HOST", "CLIENT", "TCP"), daemon=True).start()
    threading.Thread(target=send_loop, args=(conn, stop_event, "HOST", "CLIENT", "TCP"), daemon=True).start()
    stop_event.wait()

    conn.close()
    srv.close()
    print("[TCP HOST] Closed.")

def tcp_join():
    """Connects to a TCP server as a client."""
    host = input("Server host/IP [127.0.0.1]: ") or "127.0.0.1"
    port = int(input("Port [5000]: ") or 5000)

    cli = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        cli.connect((host, port))
        print("[TCP CLIENT] Connected!")

        stop_event = threading.Event()
        threading.Thread(target=receive_loop, args=(cli, stop_event, "CLIENT", "HOST", "TCP"), daemon=True).start()
        threading.Thread(target=send_loop, args=(cli, stop_event, "CLIENT", "HOST", "TCP"), daemon=True).start()
        stop_event.wait()
    except ConnectionRefusedError:
        print("[TCP CLIENT] Connection refused. Is host running?")
    finally:
        cli.close()
        print("[TCP CLIENT] Closed.")

# -------------------- UDP --------------------
def udp_host():
    """Starts a UDP server and waits for a client's first message."""
    host = input("Bind host [0.0.0.0]: ") or "0.0.0.0"
    port = int(input("Port [6000]: ") or 6000)

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((host, port))
    print(f"[UDP HOST] Listening on {host}:{port} ...")

    # Wait for the first message to discover the client's address.
    # This ensures a clean start before the main loops begin.
```

chat_Ai.py - C:\Users\Kavyansh Goenka\Downloads\CN Proj\chat_Ai.py (3.13.7)

File Edit Format Run Options Window Help

```
    print("[UDP HOST] Waiting for client to send first message...")
    data, client_addr = sock.recvfrom(BUFFER)
    print(f"\n[CLIENT] -> {data.decode().strip()}")

    stop_event = threading.Event()
    threading.Thread(target=receive_loop, args=(sock, stop_event, "HOST", "CLIENT", "UDP"), daemon=True).start()
    threading.Thread(target=send_loop, args=(sock, stop_event, "HOST", "CLIENT", "UDP", client_addr), daemon=True).start()
    stop_event.wait()
    sock.close()
    print("[UDP HOST] Closed.")

def udp_join():
    """Connects to a UDP server by sending the first message."""
    host = input("Server host/IP [127.0.0.1]: ") or "127.0.0.1"
    port = int(input("Port [6000]: ") or 6000)

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_addr = (host, port)

    print("[UDP CLIENT] Connected!")
    # Send the first message to initiate the handshake with the server.
    sock.sendto("Hiiiee".encode(), server_addr)

    stop_event = threading.Event()
    threading.Thread(target=receive_loop, args=(sock, stop_event, "CLIENT", "HOST", "UDP"), daemon=True).start()
    threading.Thread(target=send_loop, args=(sock, stop_event, "CLIENT", "HOST", "UDP", server_addr), daemon=True).start()
    stop_event.wait()
    sock.close()
    print("[UDP CLIENT] Closed.")

# -------------------- Main Menu --------------------
def main_menu():
    while True:
        print("\n===== CHAT MENU =====")
        print("1. TCP Chat")
        print("2. UDP Chat")
        print("3. Exit")
        choice = input("Enter your choice: ").strip()
        if choice == "1":
            sub = input("Host or Join? (h/j): ").strip().lower()
            if sub == "h":
                tcp_host()
            elif sub == "j":
                tcp_join()
            else:
                print("Invalid choice. Please enter 'h' or 'j'.")
        elif choice == "2":
            sub = input("Host or Join? (h/j): ").strip().lower()
            if sub == "h":
                udp_host()
            elif sub == "j":
                print("Invalid choice. Please enter 'h' or 'j'.")
        elif choice == "3":
            print("Exiting program.")
            break
        else:
            print("Invalid choice! Enter 1-3.")

if __name__ == "__main__":
    main_menu()
```

Ln: 1 Col: 0

Output:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
======== RESTART: C:\Users\Kavyansh Goenka\Downloads\CN Proj\chat_Ai.py ========

===== CHAT MENU =====
1. TCP Chat
2. UDP Chat
3. Exit
Enter your choice: 1
Host or Join? (h/j): h
Bind host [0.0.0.0]:
Port [5000]:
[TCP HOST] Listening on 0.0.0.0:5000 ...
[TCP HOST] Connected by ('127.0.0.1', 63440)
HOST -> CLIENT:

[CLIENT] -> hello
HOST -> CLIENT:

hey
HOST -> CLIENT:
quit

[CLIENT] disconnected.[TCP HOST] Closed.


===== CHAT MENU =====
1. TCP Chat
2. UDP Chat
3. Exit
Enter your choice: 2
Host or Join? (h/j): h
Bind host [0.0.0.0]:
Port [6000]:
[UDP HOST] Listening on 0.0.0.0:6000 ...
[UDP HOST] Waiting for client to send first message...

[CLIENT] -> Hiiiee
HOST -> CLIENT:

[CLIENT] -> hello
HOST -> CLIENT:

yes
HOST -> CLIENT:
quit
[UDP HOST] Closed.

===== CHAT MENU =====
1. TCP Chat
2. UDP Chat
3. Exit
Enter your choice: 3
Exiting program.
```

Ln: 56 Col: 0

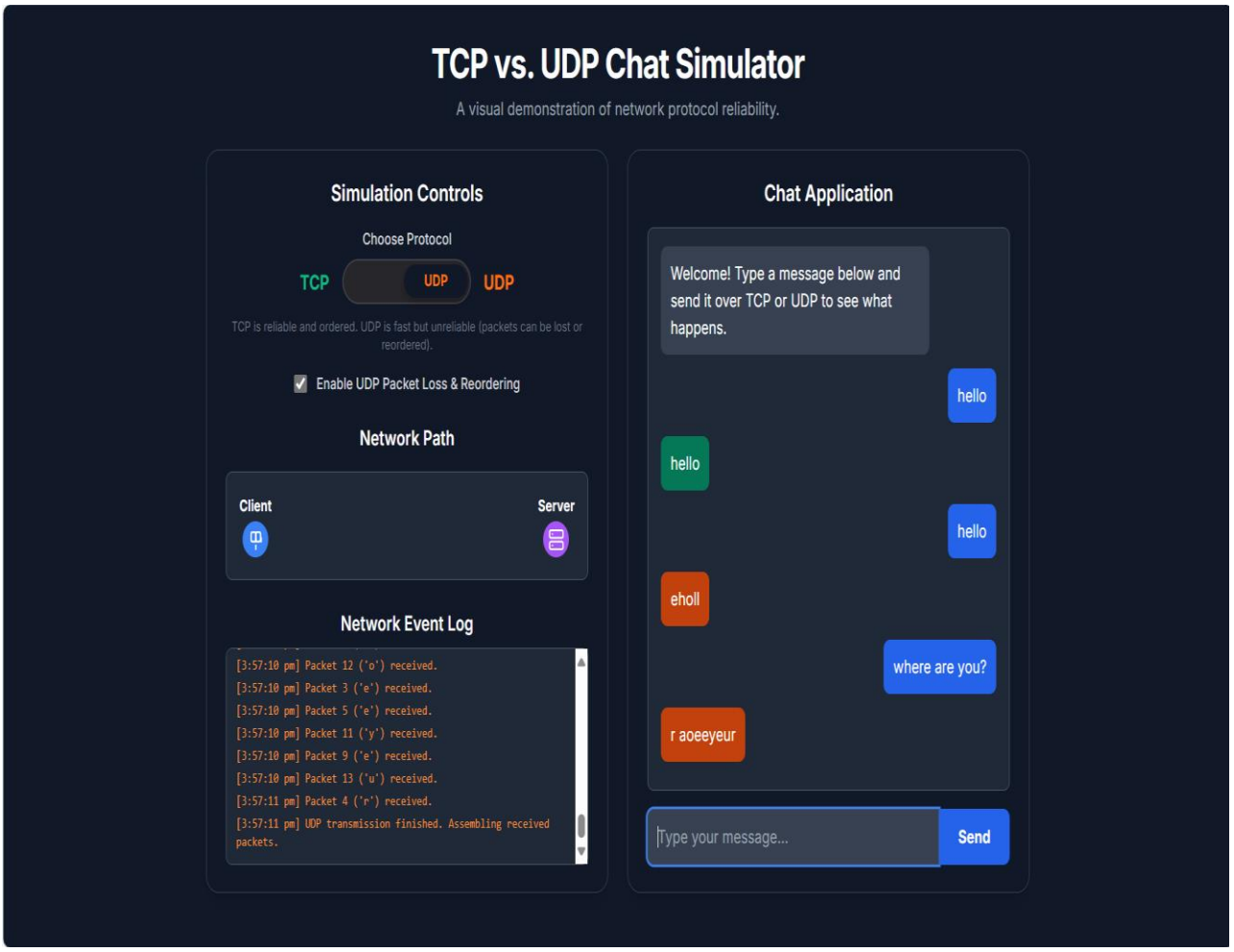
IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:\Users\Kavyansh Goenka\Downloads\CN Proj\chat_Ai.py ========

===== CHAT MENU =====
1. TCP Chat
2. UDP Chat
3. Exit
Enter your choice: 1
Host or Join? (h/j): j
Server host/IP [127.0.0.1]:
Port [5000]:
[TCP CLIENT] Connected!
CLIENT -> HOST:
hello
CLIENT -> HOST:

[HOST] -> hey
CLIENT -> HOST:

quit
[TCP CLIENT] Closed.

===== CHAT MENU =====
1. TCP Chat
2. UDP Chat
3. Exit
Enter your choice: 2
Host or Join? (h/j): j
Server host/IP [127.0.0.1]:
Port [6000]:
[UDP CLIENT] Connected!
CLIENT -> HOST:
hello
CLIENT -> HOST:

[HOST] -> yes
CLIENT -> HOST:

quit
[UDP CLIENT] Closed.

===== CHAT MENU =====
1. TCP Chat
2. UDP Chat
3. Exit
Enter your choice: 3
Exiting program.
>>>
```

Ln: 50 Col: 0

Web Application



Conclusion

In conclusion, this mini-project on the TCP vs UDP Chat Application successfully demonstrated the practical and theoretical differences between the two fundamental transport layer protocols. The project provided both real-world implementation through Python and visual simulation through HTML/JavaScript, enhancing conceptual understanding of connection-oriented and connectionless communication. It served as an effective educational tool to illustrate how reliability, sequencing, and acknowledgment in TCP differ from the lightweight and fast transmission style of UDP.

Future Scope

Future enhancements to this project may include developing a graphical user interface (GUI) for the chat application using frameworks like Tkintercad expanding support for file transfer and encryption, and deploying the simulator as a web-based learning platform accessible to students worldwide. Integrating real-time network traffic visualization and packet analysis could make it even more informative for network engineering students.

Applications

This project can be used as a learning aid in computer networking courses to help students better understand the internal workings of TCP and UDP protocols. It can also serve as a base model for developing more complex applications like VoIP systems, multiplayer games, or real-time IoT communication systems.

References

1. AI Tools – OpenAI Chatgpt, Google Gemini
2. Study materials provided by faculty professors.
3. Python Documentation - Socket Programming: https://docs.python.org/3/library/socket.html
4. W3Schools - JavaScript Tutorial: https://www.w3schools.com/js/
5. 5. GeeksforGeeks - TCP vs UDP: https://www.geeksforgeeks.org/differences-between-tcp-and-udp/
6. Google images
7. Conceptual understanding - Youtube